



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №11
по курсу «Языки и методы программирования»
«Разработка парсеров на языке Java»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Реализовать анонимный класс. Записать видео демонстрации кода и работоспособности программы, выложить в чат.

2 Задание

Овладение методом рекурсивного спуска для разработки парсеров по грамматике некоторого формального языка.

3 Результаты

Исходный код программы представлен в 1.

Листинг 1: Файл

```
1  import java.util.*;
2
3  // enum
4  enum TokenType {
5      CLASS, DEF, IDENT, LPAREN, RPAREN, COMMA, END, EOF
6  }
7
8  //
9  class Token {
10     TokenType type;
11     String text;
12     int line, col;
13
14     Token(TokenType type, String text, int line, int col) {
15         this.type = type;
16         this.text = text;
17         this.line = line;
18         this.col = col;
19     }
20 }
21
22 //
23 class Lexer {
24     private final List<String> lines;
25     private int line = 0, col = 0;
26     private String currentLine = "";
```

```

27     private int index = 0;
28
29     Lexer(Scanner scanner) {
30         lines = new ArrayList<>();
31         while (scanner.hasNextLine()) {
32             String l = scanner.nextLine();
33             if (l.isEmpty()) break; // =
34
35             lines.add(l);
36         }
37         if (!lines.isEmpty()) currentLine = lines.get(0);
38     }
39
40     //
41     Token nextToken() {
42         while (true) {
43             if (index >= currentLine.length()) {
44                 line++;
45                 if (line >= lines.size()) return new Token(TokenType
.EOF, "", line + 1, 1);
46                 currentLine = lines.get(line);
47                 index = 0;
48                 col = 0;
49                 continue;
50             }
51
52             char ch = currentLine.charAt(index);
53             col = index + 1;
54
55             if (Character.isWhitespace(ch)) {
56                 index++;
57                 continue;
58             }
59
60             if (Character.isLetter(ch)) {
61                 int start = index;
62                 while (index < currentLine.length() &&
(Character.isLetterOrDigit(currentLine.
charAt(index)))) index++;
63                 String word = currentLine.substring(start, index);
64                 switch (word) {
65                     case "class": return new Token(TokenType.CLASS,
word, line + 1, start + 1);
66                     case "def": return new Token(TokenType.DEF, word
, line + 1, start + 1);

```

```

67         case "end": return new Token(TokenType.END, word
, line + 1, start + 1);
68         default: return new Token(TokenType.IDENT, word,
        line + 1, start + 1);
69     }
70 }
71
72     if (ch == '(') { index++; return new Token(TokenType.
LPAREN, "(", line + 1, col); }
73     if (ch == ')') { index++; return new Token(TokenType.
RPAREN, ")", line + 1, col); }
74     if (ch == ',') { index++; return new Token(TokenType.
COMMA, ",", line + 1, col); }
75
76     throw new RuntimeException("syntax error at (" + (line +
1) + ", " + (col) + ")");
77 }
78 }
79 }
80
81 //
82 class Parser {
83     private Token token;
84     private final Lexer lexer;
85
86     Parser(Lexer lexer) {
87         this.lexer = lexer;
88         next();
89     }
90
91     //
92     void next() {
93         token = lexer.nextToken();
94     }
95
96     void error() {
97         throw new RuntimeException("syntax error at (" + token.line
+ ", " + token.col + ")");
98     }
99
100    // <Class> ::= class IDENT <Body>
101    void parseClass() {
102        if (token.type != TokenType.CLASS) error();
103        System.out.println("<Class> ::= class IDENT <Body>");
104        next();
105        if (token.type != TokenType.IDENT) error();

```

```

106         next ();
107         parseBody ();
108     }
109
110     // <Body> ::= <Method> <Body> | end
111     void parseBody () {
112         if (token.type == TokenType.DEF) {
113             System.out.println("<Body> ::= <Method> <Body>");
114             parseMethod ();
115             parseBody ();
116         } else if (token.type == TokenType.END) {
117             System.out.println("<Body> ::= end");
118             next ();
119         } else {
120             error ();
121         }
122     }
123
124     // <Method> ::= def IDENT <Args> end
125     void parseMethod () {
126         System.out.println("<Method> ::= def IDENT <Args> end");
127         next (); // def
128         if (token.type != TokenType.IDENT) error ();
129         next (); //
130         parseArgs ();
131         if (token.type != TokenType.END) error ();
132         next ();
133     }
134
135     // <Args> ::= ( <List> ) |
136     void parseArgs () {
137         if (token.type == TokenType.LPAREN) {
138             System.out.println("<Args> ::= ( <List> )");
139             next ();
140             parseList ();
141             if (token.type != TokenType.RPAREN) error ();
142             next ();
143         } else {
144             System.out.println("<Args> ::= ");
145         }
146     }
147
148     // <List> ::= IDENT <Tail> |
149     void parseList () {
150         if (token.type == TokenType.IDENT) {
151             System.out.println("<List> ::= IDENT <Tail>");

```

```

152         next();
153         parseTail();
154     } else {
155         System.out.println("<List> ::= ");
156     }
157 }
158
159 // <Tail> ::= , IDENT <Tail> |
160 void parseTail() {
161     if (token.type == TokenType.COMMA) {
162         System.out.println("<Tail> ::= , IDENT <Tail>");
163         next();
164         if (token.type != TokenType.IDENT) error();
165         next();
166         parseTail();
167     } else {
168         System.out.println("<Tail> ::= ");
169     }
170 }
171 }
172
173 //
174 public class Main {
175     public static void main(String[] args) {
176         Scanner scanner = new Scanner(System.in);
177         Lexer lexer = new Lexer(scanner);
178         Parser parser = new Parser(lexer);
179         try {
180             parser.parseClass();
181         } catch (RuntimeException e) {
182             System.out.println(e.getMessage());
183         }
184     }
185 }

```

Результат запуска представлен на рисунке 1.

```
/home/keiichi/.jdk/openjdk-24/bin/j
class X
def m1
end
def m2(a, b)
end
end

<Class> ::= class IDENT <Body>
<Body> ::= <Method> <Body>
<Method> ::= def IDENT <Args> end
<Args> ::=  $\epsilon$ 
<Body> ::= <Method> <Body>
<Method> ::= def IDENT <Args> end
<Args> ::= ( <List> )
<List> ::= IDENT <Tail>
<Tail> ::= , IDENT <Tail>
<Tail> ::=  $\epsilon$ 
<Body> ::= end

Process finished with exit code 0
|
```

Рис. 1 — Результат